

Retrofit + Gson 超详解：小白也能轻松上手

在Android开发中，“网络请求+数据解析”是绕不开的核心需求。比如APP加载新闻列表、用户登录注册，本质都是“向服务器要数据→把数据转成APP能看懂的格式”。

原生的网络请求（如HttpURLConnection）代码又长又容易出错，数据解析还要手动写一堆代码。而 **Retrofit + Gson** 的组合，就像给开发者配了“全自动工具包”——Retrofit帮你搞定网络请求的繁琐操作，Gson帮你秒转服务器数据，两者搭配堪称“黄金搭档”。

本文全程避开复杂术语，用“大白话+实操代码”带你吃透这套组合，看完就能动手写。

一、先搞懂：Retrofit和Gson分别是啥？

在学用法之前，先明确两者的定位，避免越学越乱。

1. Retrofit：网络请求的“指挥官”

Retrofit本身**不直接做网络请求**，它是基于OkHttp（一个强大的网络库）封装的“指挥官”——你只需要告诉它“请求哪个接口、传什么参数”，它就会调用OkHttp完成底层的连接、发送、接收操作，最后把结果给你。

核心优势：用“注解”代替繁琐代码（比如@GET表示获取数据，@POST表示提交数据），自动处理子线程（不用你手动开线程），还能和Gson无缝配合。

2. Gson：数据解析的“翻译官”

服务器返回的数据几乎都是**JSON格式**（比如{"name":"张三","age":20}），这种格式计算机能懂，但Java代码没法直接用（Java需要用对象来操作数据）。

Gson就是把JSON字符串“翻译”成Java对象的“翻译官”，也能把Java对象“翻译”成JSON字符串发给服务器，全程自动，不用你手动截取字符串。

3. 两者的关系：分工合作，无缝衔接

举个例子：你要做“获取用户信息”的功能

1. 你通过Retrofit发起请求，告诉它“用户ID是1001”；
2. Retrofit调用OkHttp向服务器发送请求；
3. 服务器返回JSON数据：{"id":1001,"name":"张三","age":20}；
4. Retrofit自动把JSON交给Gson；
5. Gson把JSON转成Java对象（比如User对象，里面有id、name、age属性）；
6. 你直接用User对象获取数据（user.getName()就能拿到“张三”）。

二、前期准备：环境配置（3步搞定）

在写代码前，需要先把Retrofit和Gson的“工具”导入项目，就像做饭前先准备好锅碗瓢盆。

1. 添加网络权限

所有网络操作都需要权限，在AndroidManifest.xml里添加（注意放在<application>标签外面）：

Code block

```
1  <!-- 网络权限：必须加 -->
2  <uses-permission android:name="android.permission.INTERNET" />
3
4
5  <application
6      ...
7      android:usesCleartextTraffic="true">
8      ...
9  </application>
```

2. 导入依赖

在app模块下的build.gradle（Module :app）里添加依赖，同步后就能使用Retrofit和Gson了。

Code block

```
1  dependencies {
2      // 1. Retrofit核心库
3      implementation 'com.squareup.retrofit2:retrofit:2.9.0'
4      // 2. Gson转换器（Retrofit和Gson的桥梁）
5      implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
6      // 3. OkHttp（Retrofit依赖它，可选：如果需要自定义配置才加）
7      implementation 'com.squareup.okhttp3:okhttp:4.11.0'
8  }
```

 同步方法：添加依赖后，点击Android Studio右上角的“Sync Now”，等待同步完成（如果报错，检查网络是否正常，或刷新依赖）。

3. 确认实体类规则（关键！）

Gson在“翻译”JSON时，需要一个“模板”——也就是Java实体类，这个类的属性名必须和JSON的键名一致（大小写也要一致）。

比如服务器返回的JSON是：

```

1  {
2      "code": 200,
3      "msg": "success",
4      "data": {
5          "id": 1001,
6          "name": "张三",
7          "age": 20,
8          "gender": "男"
9      }
10 }

```

对应的实体类就要这样写（可以手动写，也可以用插件自动生成，后面会说）：

Code block

```

1  // 最外层的响应实体类（服务器返回的所有数据都包裹在这里）
2  public class BaseResponse<T> {
3      // 和JSON的"code"对应
4      private int code;
5      // 和JSON的"msg"对应
6      private String msg;
7      // 和JSON的"data"对应（T是泛型，可接收任意类型，比如User、List<News>）
8      private T data;
9
10     // 必须有getter和setter方法（用于获取和设置数据）
11     public int getCode() { return code; }
12     public void setCode(int code) { this.code = code; }
13     public String getMsg() { return msg; }
14     public void setMsg(String msg) { this.msg = msg; }
15     public T getData() { return data; }
16     public void setData(T data) { this.data = data; }
17 }
18
19 // 内层的User实体类（对应JSON里的data对象）
20 public class User {
21     private int id;
22     private String name;
23     private int age;
24     private String gender;
25
26     // getter和setter方法
27     public int getId() { return id; }
28     public void setId(int id) { this.id = id; }
29     public String getName() { return name; }
30     public void setName(String name) { this.name = name; }
31     public int getAge() { return age; }
32     public void setAge(int age) { this.age = age; }

```

```
33     public String getGender() { return gender; }
34     public void setGender(String gender) { this.gender = gender; }
35 }
```

 **重点：**实体类的属性名必须和JSON的键名完全一致！如果JSON里是“userName”，实体类就不能写“username”，否则Gson会“翻译错误”，返回null。

三、核心步骤：Retrofit + Gson 完整流程（以3个常用场景为例）

Retrofit的核心用法可以总结为“3步走”：**定义接口→创建Retrofit实例→发起请求**。下面结合“获取数据（GET）、提交数据（POST）、上传图片”三个最常用的场景，一步步教你写代码。

先统一说明：本文用“模拟接口”演示，实际开发中替换成你们后端给的接口地址即可。

场景1：GET请求——获取用户信息（根据ID查用户）

需求：向接口发送用户ID，获取该用户的详细信息（id、name、age等）。

步骤1：定义API接口（告诉Retrofit怎么请求）

创建一个接口类（比如叫ApiService），用“注解”描述请求信息——这是Retrofit的核心特色，不用写一行网络连接代码。

Code block

```
1  import retrofit2.Call;
2  import retrofit2.http.GET;
3  import retrofit2.http.Query;
4
5  // API接口类：所有网络请求都在这里定义
6  public interface ApiService {
7      /**
8       * GET请求：获取用户信息
9       * @GET("user/getInfo")：表示这是GET请求，接口路径是"user/getInfo"
10      * @Query("userId")：表示请求参数的键是"userId"，值是传入的int类型参数
11      * Call<BaseResponse<User>>：表示请求成功后，返回BaseResponse<User>类型的数据
12      */
13      @GET("user/getInfo")
14      Call<BaseResponse<User>> getUserInfo(@Query("userId") int userId);
15 }
```

注解说明：

- @GET("user/getInfo"): 第一个参数是“接口相对路径”，完整接口地址=“基础URL”+“相对路径”（比如基础URL是<http://xxx.xxx.xxx/api/>，完整地址就是<http://xxx.xxx.xxx/api/user/getInfo>）；
- @Query("userId"): 用于传递GET请求的参数（拼在URL后面，比如?userId=1001）；
- Call<BaseResponse<User>>: Retrofit的“请求对象”，泛型里是服务器返回的数据类型（这里是BaseResponse包裹的User对象）。

步骤2：创建Retrofit实例（配置基础信息）

在Activity或ViewModel里，创建Retrofit实例，配置“基础URL”和“Gson转换器”（告诉Retrofit用Gson解析数据）。

Code block

```

1  import retrofit2.Retrofit;
2  import retrofit2.converter.gson.GsonConverterFactory;
3
4  public class MainActivity extends AppCompatActivity {
5      // 1. 基础URL (后端给的, 注意末尾要加"/")
6      private static final String BASE_URL = "http://xxx.xxx.xxx/api/";
7      // 2. API接口实例
8      private ApiService apiService;
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14
15         // 3. 创建Retrofit实例
16         Retrofit retrofit = new Retrofit.Builder()
17             .baseUrl(BASE_URL) // 配置基础URL
18             .addConverterFactory(GsonConverterFactory.create()) // 配置Gson
19             .build();
20
21         // 4. 获取API接口实例 (Retrofit自动生成实现类)
22         apiService = retrofit.create(ApiService.class);
23
24         // 5. 调用方法发起请求 (下文写)
25         getUsersInfo(1001);
26     }
27 }
```

步骤3：发起请求并处理结果

Retrofit支持“异步请求”（推荐，不会卡主线程）和“同步请求”（需手动开子线程），这里用异步请求（调用enqueue方法）。

Code block

```
1  // 发起GET请求：获取用户信息
2  private void getUsersInfo(int userId) {
3      // 1. 调用接口方法，得到请求对象
4      Call<BaseResponse<User>> call = apiService.getUserInfo(userId);
5
6      // 2. 发起异步请求（enqueue是异步，execute是同步）
7      call.enqueue(new retrofit2.Callback<BaseResponse<User>>() {
8          // 3. 请求成功（服务器返回数据）
9          @Override
10         public void onResponse(Call<BaseResponse<User>> call,
11             retrofit2.Response<BaseResponse<User>> response) {
12             // 检查响应是否成功（code=200表示HTTP请求成功）
13             if (response.isSuccessful()) {
14                 // 获取服务器返回的完整数据（BaseResponse对象）
15                 BaseResponse<User> baseResponse = response.body();
16                 // 检查业务是否成功（比如后端用code=200表示业务成功）
17                 if (baseResponse != null && baseResponse.getCode() == 200) {
18                     // 获取User对象（Gson已自动解析完成）
19                     User user = baseResponse.getData();
20                     // 用User对象获取数据（更新UI）
21                     String userName = user.getName();
22                     int userAge = user.getAge();
23                     Toast.makeText(MainActivity.this, "姓名：" + userName + ",
24                         年龄：" + userAge, Toast.LENGTH_SHORT).show();
25                 } else {
26                     // 业务失败（比如用户ID不存在）
27                     Toast.makeText(MainActivity.this, "获取失败：" +
28                         baseResponse.getMsg(), Toast.LENGTH_SHORT).show();
29                 }
30             }
31         }
32     });
33
34     // 4. 请求失败（比如网络中断、接口不存在）
35     @Override
36     public void onFailure(Call<BaseResponse<User>> call, Throwable t) {
37         Toast.makeText(MainActivity.this, "请求失败：" + t.getMessage(),
38             Toast.LENGTH_SHORT).show();
39     }
40 }
```

 关键说明：onResponse和onFailure都是在子线程吗？不是！Retrofit已经帮你切换到主线程了，所以可以直接在里面更新UI（比如弹Toast、改TextView），不用手动写runOnUiThread。

场景2：POST请求——用户登录（提交账号密码）

需求：向服务器提交用户名和密码，验证成功后返回用户信息。POST请求的参数是放在“请求体”里的，比GET更安全（不会暴露在URL上）。

步骤1：修改API接口（添加POST方法）

Code block

```
1  public interface ApiService {
2      // 之前的GET请求方法...
3
4      /**
5       * POST请求：用户登录
6       * @FormUrlEncoded：表示参数是表格式（类似网页表单提交）
7       * @POST("user/login")：POST请求，接口路径
8       * @Field("username")：表单参数的键，值是传入的字符串
9       */
10     @FormUrlEncoded
11     @POST("user/login")
12     Call<BaseResponse<User>> userLogin(
13         @Field("username") String username,
14         @Field("password") String password
15     );
16 }
```

步骤2：发起POST请求（类似GET）

Code block

```
1  // 发起POST请求：用户登录
2  private void userLogin(String username, String password) {
3      Call<BaseResponse<User>> call = apiService.userLogin(username, password);
4      call.enqueue(new retrofit2.Callback<BaseResponse<User>>() {
5          @Override
6          public void onResponse(Call<BaseResponse<User>> call,
7              retrofit2.Response<BaseResponse<User>> response) {
8              if (response.isSuccessful()) {
9                  BaseResponse<User> baseResponse = response.body();
10                 if (baseResponse != null && baseResponse.getStatusCode() == 200) {
11                     User user = baseResponse.getData();
12                 }
13             }
14         }
15     });
16 }
```

```

11             Toast.makeText(MainActivity.this, "登录成功, 欢迎: " +
user.getName(), Toast.LENGTH_SHORT).show();
12         } else {
13             Toast.makeText(MainActivity.this, "登录失败: " +
baseResponse.getMsg(), Toast.LENGTH_SHORT).show();
14         }
15     }
16 }
17
18 @Override
19 public void onFailure(Call<BaseResponse<User>> call, Throwable t) {
20     Toast.makeText(MainActivity.this, "网络异常: " + t.getMessage(),
Toast.LENGTH_SHORT).show();
21 }
22 });
23 }

```

调用方法：在onCreate里添加userLogin("test", "123456")即可。

场景3：POST请求——上传图片（附带普通参数）

需求：上传本地图片到服务器，同时提交用户ID（普通参数）。图片上传需要用“Multipart”格式，Retrofit也支持这种场景。

步骤1：添加权限（图片上传需要读取本地文件）

在AndroidManifest.xml里添加读写权限，并且Android 6.0+需要动态申请权限（这里简化，先加静态权限，动态权限可自行补充）：

Code block

```

1 <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
2 <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

```

步骤2：修改API接口（添加图片上传方法）

Code block

```

1 import okhttp3.MultipartBody;
2 import okhttp3.RequestBody;
3 import retrofit2.Call;
4 import retrofit2.http.Multipart;
5 import retrofit2.http.POST;
6 import retrofit2.http.Part;
7
8 public interface ApiService {

```



```

9      // 之前的GET、POST方法...
10
11     /**
12      * POST请求：上传图片
13      * @Multipart：表示请求是多部分格式（支持文件+普通参数）
14      * @Part("userId")：普通参数（用RequestBody包装）
15      * @Part MultipartBody.Part：图片文件参数（name是"image"，后端用这个键接收）
16      */
17     @Multipart
18     @POST("file/uploadImage")
19     Call<BaseResponse<String>> uploadImage(
20         @Part("userId") RequestBody userId,
21         @Part MultipartBody.Part image
22     );
23 }

```

步骤3：封装参数并发起请求

图片需要转成MultipartBody.Part格式，普通参数需要转成RequestBody格式。

Code block

```

1  import okhttp3.MediaType;
2  import okhttp3.MultipartBody;
3  import okhttp3.RequestBody;
4  import java.io.File;
5
6  // 上传图片
7  private void uploadImage(String userId, String imagePath) {
8      // 1. 封装普通参数 (userId)
9      RequestBody userIdBody = RequestBody.create(
10         MediaType.parse("text/plain"), // 参数类型（文本）
11         userId // 参数值
12     );
13
14     // 2. 封装图片文件
15     File imageFile = new File(imagePath); // 图片本地路径（比
    如/sdcard/Pictures/test.jpg）
16     // 图片的MediaType（image/jpeg表示JPG格式，image/png表示PNG格式）
17     RequestBody imageBody = RequestBody.create(MediaType.parse("image/jpeg"),
    imageFile);
18     // 封装成MultipartBody.Part（name="image"是后端约定的键）
19     MultipartBody.Part imagePart = MultipartBody.Part.createFormData(
20         "image", // 后端接收图片的键
21         imageFile.getName(), // 图片文件名
22         imageBody
23     );

```

```

24
25     // 3. 发起请求
26     Call<BaseResponse<String>> call = apiService.uploadImage(userIdBody,
imagePart);
27     call.enqueue(new retrofit2.Callback<BaseResponse<String>>() {
28         @Override
29         public void onResponse(Call<BaseResponse<String>> call,
retrofit2.Response<BaseResponse<String>> response) {
30             if (response.isSuccessful()) {
31                 BaseResponse<String> baseResponse = response.body();
32                 if (baseResponse != null && baseResponse.getCode() == 200) {
33                     // 服务器返回图片的URL (比如
http://xxx.xxx.xxx/images/123.jpg)
34                     String imageUrl = baseResponse.getData();
35                     Toast.makeText(MainActivity.this, "上传成功, 图片URL: " +
imageUrl, Toast.LENGTH_SHORT).show();
36                 } else {
37                     Toast.makeText(MainActivity.this, "上传失败: " +
baseResponse.getMsg(), Toast.LENGTH_SHORT).show();
38                 }
39             }
40         }
41
42         @Override
43         public void onFailure(Call<BaseResponse<String>> call, Throwable t) {
44             Toast.makeText(MainActivity.this, "上传失败: " + t.getMessage(),
Toast.LENGTH_SHORT).show();
45         }
46     });
47 }

```

调用方法：uploadImage("1001", "/sdcard/Pictures/test.jpg")（注意图片路径要真实存在）。

四、进阶技巧：让开发更高效

1. 自动生成实体类（告别手动写）

手动写实体类容易出错，尤其是JSON结构复杂的时候。可以用Android Studio插件“GsonFormat”自动生成：

1. 安装插件：File → Settings → Plugins → 搜索“GsonFormat” → 安装并重启AS；
2. 新建空类（比如User），右键 → Generate → GsonFormat；
3. 把JSON字符串粘贴进去，点击OK，自动生成属性和getter/setter方法。

2. 统一管理Retrofit实例（单例模式）

如果每个页面都创建Retrofit实例，会浪费资源。可以用单例模式封装一个Retrofit工具类：

Code block

```
1  public class RetrofitUtils {
2      private static Retrofit retrofit;
3      private static final String BASE_URL = "http://xxx.xxx.xxx/api/";
4
5      // 单例模式：获取Retrofit实例
6      public static Retrofit getRetrofit() {
7          if (retrofit == null) {
8              synchronized (RetrofitUtils.class) {
9                  if (retrofit == null) {
10                     retrofit = new Retrofit.Builder()
11                         .baseUrl(BASE_URL)
12                         .addConverterFactory(GsonConverterFactory.create())
13                         .build();
14                 }
15             }
16         }
17         return retrofit;
18     }
19
20     // 获取API接口实例
21     public static ApiService getApiService() {
22         return getRetrofit().create(ApiService.class);
23     }
24 }
```

使用时直接调用：ApiService apiService = RetrofitUtils.getApiService()，不用重复创建Retrofit。

3. 处理请求头（比如Token认证）

很多接口需要在请求头里加Token（用户登录后的身份凭证），可以在API接口里用@Headers注解添加：

Code block

```
1  // 单个请求头
2  @Headers("Authorization: Bearer " + "你的Token")
3  @GET("user/getInfo")
4  Call<BaseResponse<User>> getUserInfo(@Query("userId") int userId);
5
6  // 多个请求头
7  @Headers({
8      "Authorization: Bearer " + "你的Token",
9      "Content-Type: application/json"
```

```
10    })
11    @POST("user/login")
12    Call<BaseResponse<User>> userLogin(...);
```

如果所有接口都需要加Token，推荐用OkHttp的拦截器统一添加（避免每个接口都写一次）。

五、常见问题：小白踩坑指南

1. 请求成功但data为null?

原因：实体类属性名和JSON键名不一致。比如JSON是“user_name”，实体类写了“userName”。

解决：要么修改实体类属性名，要么用@SerializedName注解指定JSON键名：

Code block

```
1  public class User {
2      // JSON里是"user_name", 实体类用"userName", 通过注解关联
3      @SerializedName("user_name")
4      private String userName;
5      // ...
6  }
```

2. 报错“NetworkOnMainThreadException”?

原因：在主线程发起了同步请求（调用了execute方法）。

解决：改用异步请求（enqueue方法），或在子线程里调用execute。

3. 图片上传失败，提示“文件不存在”?

原因：图片路径错误，或没有获取动态权限。

解决：1. 检查图片路径是否正确；2. Android 6.0+ 用ActivityCompat.requestPermissions申请读写权限。

4. 接口地址正确，但提示“404 Not Found”?

原因：基础URL或相对路径写错。比如基础URL末尾没加“/”，相对路径又加了“/”，导致拼接后出现重复“//”。

解决：确保基础URL末尾加“/”，相对路径开头不加“/”（比如基础URL是<http://xxx/api/>，相对路径是user/getInfo）。

六、总结：Retrofit + Gson 核心优势

1. **代码简洁**：用注解代替原生网络请求的繁琐代码，几行注解就搞定请求配置；
2. **自动解析**：Gson自动把JSON转成Java对象，不用手动处理字符串；
3. **线程安全**：异步请求自动切换线程，不用手动开线程和切换主线程；
4. **扩展性强**：支持文件上传下载、请求拦截、缓存等高级功能，满足实际项目需求。

对于小白来说，掌握本文的3个场景（GET、POST、图片上传），就能应对90%以上的网络需求。实际开发中，只要把接口地址和实体类换成你们项目的，就能直接用啦！

（注：文档部分内容可能由 AI 生成）